

IDW developer implementation guide

Oct 1, 2026 · @Sunitha

The compliance rules for IndiaDentalWorld already exist as a tested PostgreSQL layer. Your job is to build the Next.js application on top of it without bypassing those rules, and to harden it for production.

What exists and what you build

Delivered (package `idw-platform`): PostgreSQL schema, rule functions, seed values and 60 automated checks in `db/`, plus a README. The tests pass on PostgreSQL 16. No application code exists yet.

You build: the Next.js application, authentication, file uploads, payments, WhatsApp messaging, hosting and the security hardening below. The HTML prototypes (homepage, search results, clinic profile, clinic registration) are the visual reference for every screen.

The database is the source of truth for compliance. If a screen and a database rule disagree, the database wins and the screen changes.

Architecture and stack

- **App:** Next.js 14 or newer (App Router), TypeScript, Tailwind CSS.
- **Database:** PostgreSQL on AWS RDS in Mumbai (ap-south-1), encrypted storage, automatic backups, no public access.
- **Data access:** Prisma for ordinary reads and writes. Keep the SQL files as the source of truth: run them as migrations, then use `prisma db pull` to generate the Prisma schema. Do not let `prisma migrate` rewrite the functions and triggers.
- **Compliance calls:** call the database functions with Prisma's raw queries, for example `SELECT record_consent($1, $2, $3, $4, $5)`.
- **Files:** private S3 bucket in Mumbai for registration documents, accessed only through short-lived signed URLs.
- **Payments:** Razorpay. **Messaging:** WhatsApp Business API. **AI search:** Claude API (see the cross-border note in the open decisions).
- **Design:** follow the prototypes. Dark first, colours `#0A1714`, `#00C4A7`, `#F06A6A`, `#F5A623`, `#34D399`; fonts Plus Jakarta Sans, Lora and JetBrains Mono.

Build order

Do the steps in this order. Each has a test you can run, so the founder can see progress.

#	Work	Done when
1	Repository, CI, environments. Run <code>db/run_tests.sh</code> on every pull request.	CI is green and fails if any compliance check fails.
2	Hosting and database roles. Create RDS in Mumbai. Create two database roles: <code>app_public</code> (patient and clinic features) and <code>app_admin</code> (admin and verifier features). Grant <code>EXECUTE</code> on each function only to the role that needs it; give neither role <code>UPDATE</code> or <code>DELETE</code> on <code>consents</code> , <code>audit_log</code> or <code>notice_versions</code> .	An <code>app_public</code> connection cannot call <code>decide_verification</code> or <code>execute_data_deletion</code> .
3	Sign-in with phone OTP, sessions, and role from <code>users.role</code> . Rate-limit OTP requests.	The user id passed to every database function comes from the session, never from the request body.
4	Consent screens and preferences page: <code>record_consent</code> , <code>withdraw_consent</code> , <code>view_current_consents</code> . Show the notice text from <code>notice_versions</code> .	Withdrawing is one click and takes effect at once.
5	Patient dashboard: download and delete my data (<code>request_data_export</code> , <code>request_data_deletion</code>), consent preferences.	A test patient can request deletion and sees its status.
6	Grievance form and officer inbox (<code>submit_grievance</code> , <code>resolve_grievance</code>). Link the form from every page footer.	A grievance gets a deadline and appears in the officer's inbox.
7	Clinic, lab, vendor and institute registration. Upload the registration document to private S3, then call <code>submit_verification</code> .	The document is not reachable by public URL.

#	Work	Done when
8	Admin queues: verification (<code>claim_verification</code> , <code>decide_verification</code>), data requests (<code>fulfil_data_export</code> , <code>execute_data_deletion</code>), review moderation (<code>moderate_review</code>). Run <code>expire_verifications()</code> daily.	Only users with the <code>admin</code> or <code>verifier</code> role reach these pages.
9	Reviews: form after a completed appointment, clinic reply, hide through moderation.	An attempt to review without a completed visit shows a clear error.
10	Badge and listing display from the view <code>verified_organisations</code> .	An expired clinic loses its badge with no code change.
11	Revenue switches: build commission, lead-fee and sponsored features behind <code>feature_flags</code> . Leave them off.	Nothing paid is visible until an admin enables the flag with a named approver.
12	Admin audit page that shows <code>audit_log</code> and runs <code>verify_audit_chain()</code> .	The check returns no rows.
13	Hardening (next section), policy pages, load and penetration testing.	Security reviewer signs off.

Rules the app must never bypass

- Never write to `consents` , `audit_log` or `notice_versions` directly. Use `record_consent` and `withdraw_consent` . New notice wording is a new `notice_versions` row, never an edit.
- Never take a user id, reviewer id or admin id from the request. Read it from the authenticated session.
- Ask for location only after the user taps the location control and `location` consent is recorded. Save search history only if `has_consent(user, 'search_history')` is true.
- Show the badge only for organisations in `verified_organisations` . Never compute "verified" in application code.
- Show the "Sponsored" label from the database row, at least 12px, on every sponsored card. Never render more than two sponsored cards on a results page.

- No sponsored content, third-party ad scripts or tracking pixels in the booking flow. No third-party script may add more than 200 ms to page load.
- Never log phone numbers, email addresses, search text or review text. Log ids only.
- Never sell or share personal data with advertisers. Revenue features stay behind `feature_flags` until an admin records approval.
- Do not use words such as "top", "best" or "most trusted" in any copy or generated text.

Security work still open

These are known gaps in the delivered database layer. Close them before real data is loaded.

1. **Make the functions run with the owner's rights.** The functions currently run with the caller's rights, so the app's database role would need direct write access to the tables. Change the functions to `SECURITY DEFINER` with a fixed `search_path`, then revoke direct table writes from `app_public` and `app_admin`. Re-run the tests under those roles.
2. **Do not trust identity arguments.** The functions accept an actor id as a parameter. Until step 1 is done, a compromised app could pass any id. Admin functions must be reachable only from the admin database role and admin routes.
3. **Row-level security.** Add policies so that a patient can read only their own rows and a clinic only its own organisation.
4. **Encrypt phone and email columns**, with the key held in AWS KMS.
5. **Uploads:** private bucket, signed URLs that expire, file type and size limits, virus scan.
6. **Sign-in abuse:** OTP rate limits, lockouts, and protection against number enumeration.
7. **Backups and retention:** automatic backups, and a rule so that a restored backup does not bring back deleted personal data.
8. **Breach procedure:** a written plan for who is told, and how fast. Counsel sets the duties.
9. **Independent review:** a security engineer reviews the database and the application, and runs a penetration test.

Acceptance checklist

The founder signs off when each item below is shown working on a staging site with sample data.

- ☐ `db/run_tests.sh` passes in CI, and also passes when run under the restricted database roles.
- ☐ A patient can give, see and withdraw each consent in one step, and the history is kept.
- ☐ A patient can download their data, and can request deletion and see the status.
- ☐ After deletion, name, phone, email and review text are gone, and the same phone number can sign up again.
- ☐ A grievance shows a deadline, appears in the officer's inbox and can be resolved.
- ☐ A clinic can upload a registration, a verifier can approve or reject it with a reason, and the badge appears and later expires.
- ☐ A user cannot approve their own verification request, and cannot call admin pages or functions.
- ☐ A review needs a completed visit; nobody can edit or delete it; an admin can hide it with a reason.
- ☐ No sponsored card, commission or lead fee appears while its flag is off.
- ☐ Every page has a Grievance Officer link and the price disclaimer; every article shows the author's qualification.
- ☐ The audit page shows the history and `verify_audit_chain()` returns no rows.
- ☐ Logs contain no phone numbers, emails, search text or review text.
- ☐ A security engineer has reviewed the system, and the founder has the lawyer's written approval of the notices.

Decisions still waiting

These values are placeholders in the database. Do not hard-code them; they are settings.

Decision	Owner	Where it lives
Final wording of the privacy notice, terms, location, search history and marketing consents	Lawyer	<code>notice_versions</code> (add a new version)
Response deadlines for data requests (30 days) and grievances (15 days)	Lawyer	<code>settings</code>
Badge validity (365 days) and how often registrations are re-checked	Founder and lawyer	<code>settings</code>

Decision	Owner	Where it lives
What deletion must keep: consent history, ratings, appointments, audit trail	Lawyer	<code>execute_data_deletion</code>
Whether booking commission, the WhatsApp lead fee and sponsored placement are allowed	Lawyer	<code>feature_flags</code>
Whether sending search text to the Claude API counts as a cross-border transfer, and what to disclose	Lawyer	AI search feature
Company details, CIN, GST and Grievance Officer name for the footer and policies	Founder	Footer and policy pages
Pricing for labs, vendors and institutes	Founder	Not yet defined